UNIVERSITY OF GOTHENBURG

Automatization – Macro Recording

Introductory Course to Image Analysis in Life Science

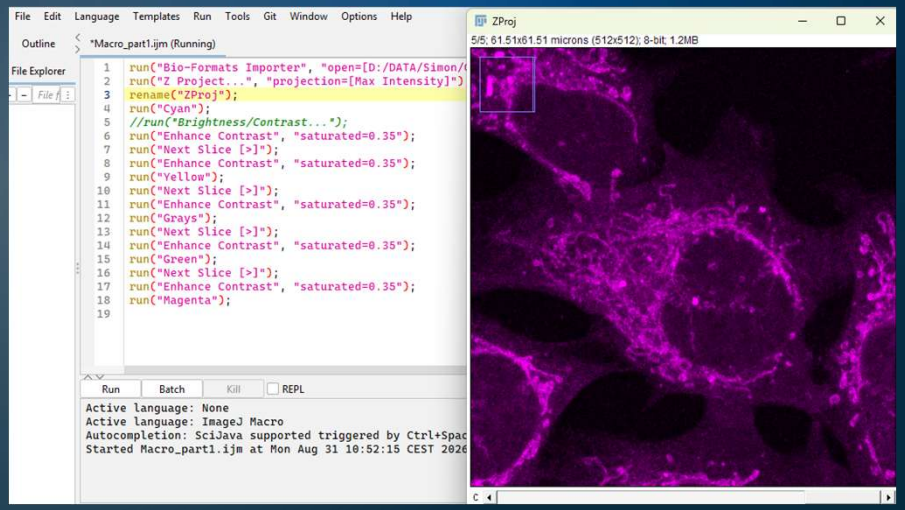
LECLERC SIMON

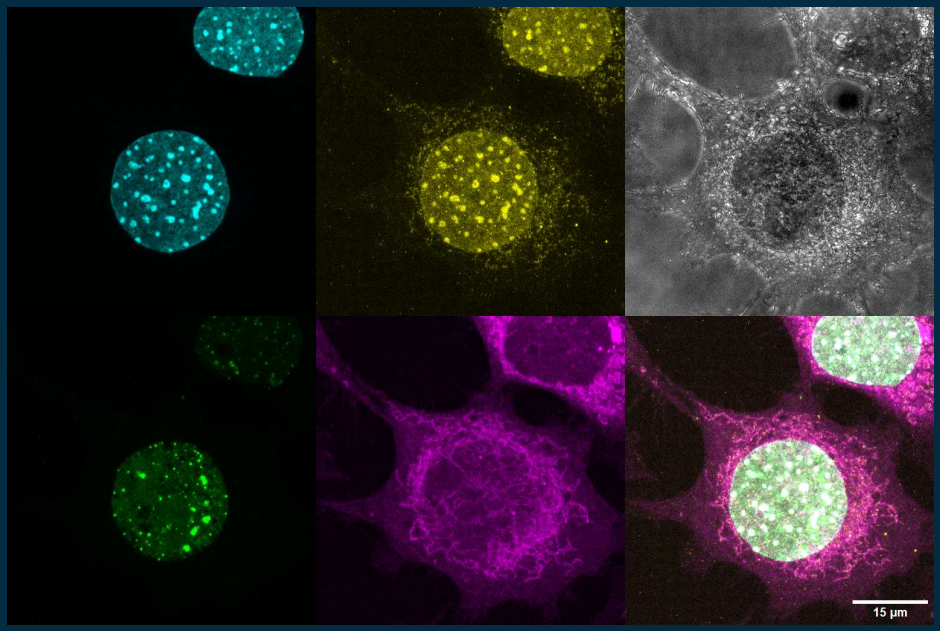
Life Line

- https://imagej.net/ij/
- https://imagej.net/ij/developer/macro/macros.html
- https://imagej.net/ij/developer/macro/functions.html

Material for the exercices:

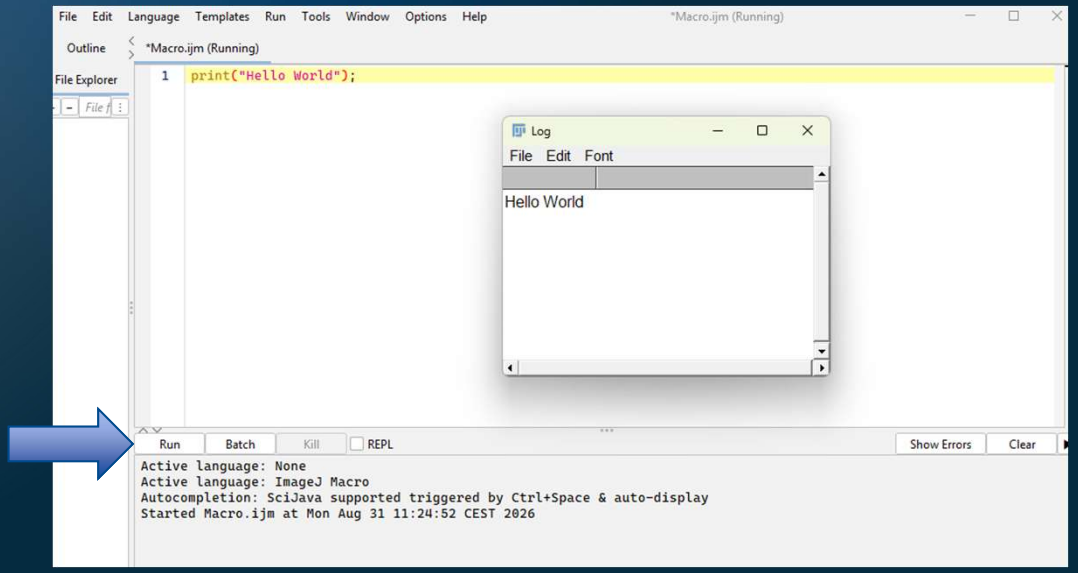
Making use of Macros to automatize image processing





The Basics

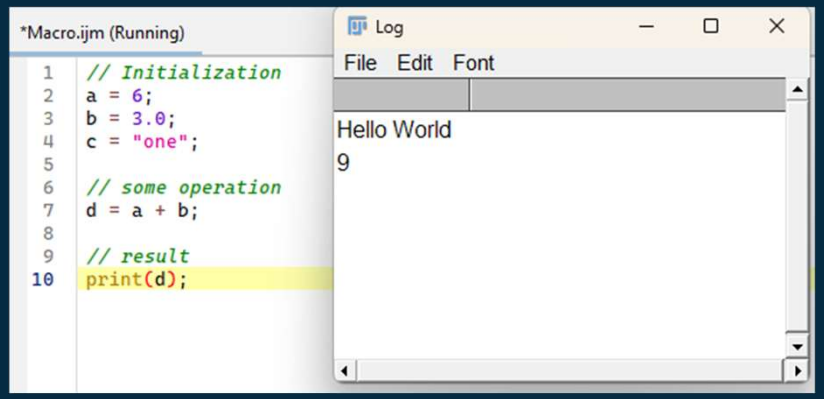
Open the Macros Editor: Plugin > New > Macro

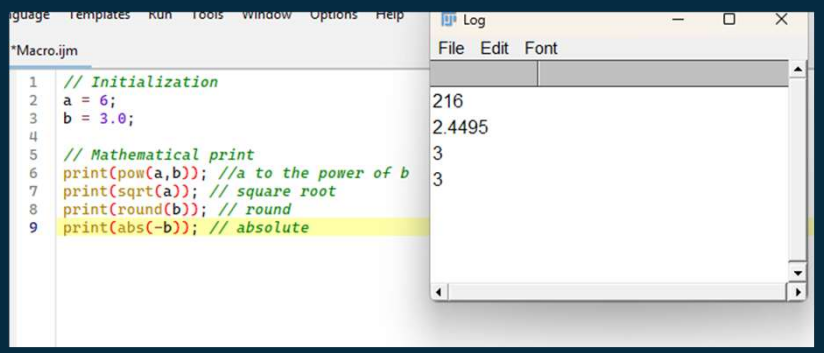


The Basics: Variables

All variables needs to be declared

A lot of mathematical functions are there



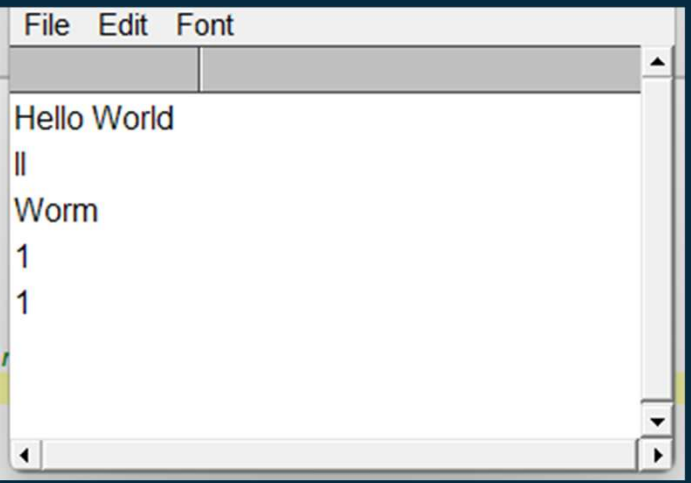


The Basics: String Manipulation

Concatenate, select a substring, replace a part of the string, check if a string starts or ends with a selection

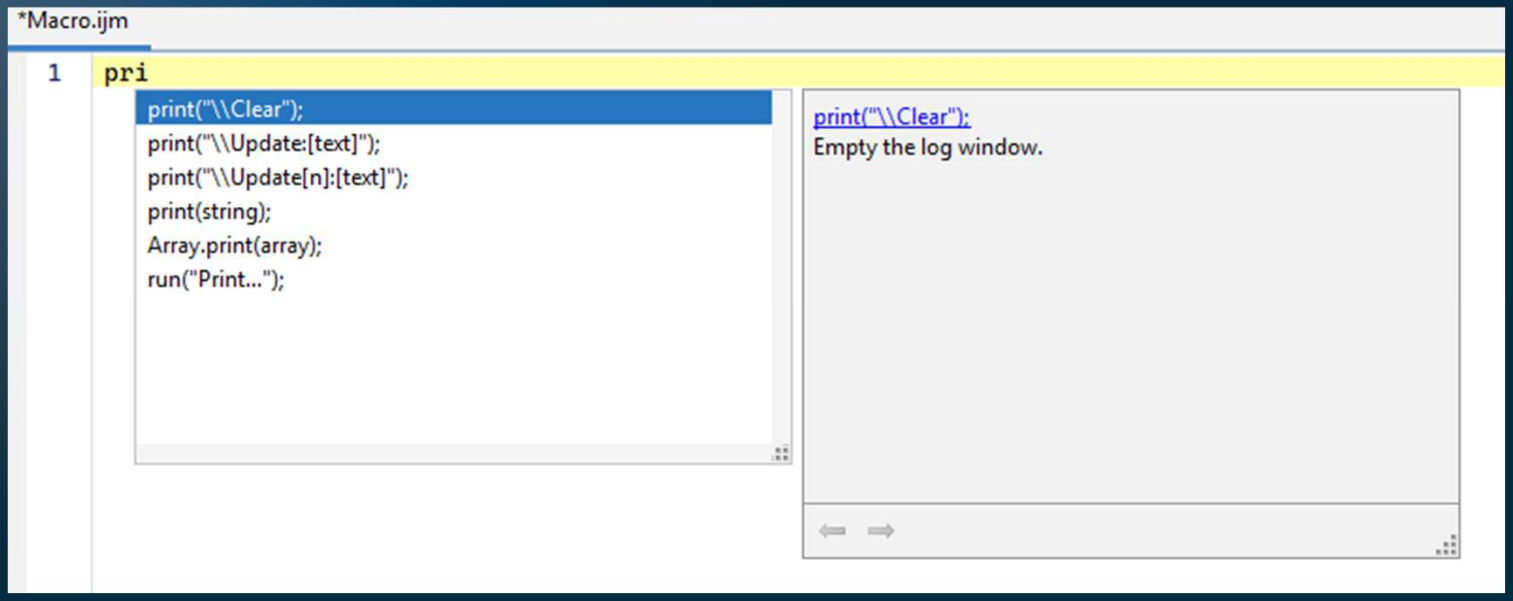
Macro.ijm

```
1  // Initialization
2  a = "Hello";
3  b = "World";
4
5  // string manipulation print
6  print(a+" "+b); // concatenate
7  print(substring(a, 2, 4)); // substring
8  print(replace(b, "ld", "m")); // replace
9  print(startsWith(a, "He")); // check the start of the stri
10 print(endsWith(b, "ld")); // check the send of the string
```



Autocompletion

Use the Autocompletion to simplify your life!



The Recorder

Easiest way to do a Macro: Record the actions that you do in Fiji!

Open it with: Plugins > Macros > Record…

- Open the Blobs image (File > Open Sample > Blobs)
- Run a Gaussian Blur with sigma of 3 (Process > Gaussian Blur...)
- Set a Otsu threshold (Image > Adjust > Threshold...)
- Set the parameters to measure as area, centroid and shape (Analyze > Set Measurements...)
- Analyze the segmented image (Analyze > Analyze Particle...)
- Then on the Recorder, click on Create to automatically generate the macro

The Recorder

```
1  run("Blobs (25K)");
2  run("Gaussian Blur...", "sigma=3");
3  setAutoThreshold("Otsu");
4  //run("Threshold...");
5  //setThreshold(123, 255);
6  setOption("BlackBackground", true);
7  run("Convert to Mask");
8  run("Close");
9  run("Set Measurements...", "area centroid shape redirect=None decimal=3");
10 run("Analyze Particles...", "size=0-Infinity display clear");
11
```

Line starting by '//' are comments, piece of text that will NOT be executed

The Recorder

```
1  //open the image
2  run("Blobs (25K)");
3
4  //processing
5  run("Gaussian Blur...", "sigma=3");
6
7  //Segmentation
8  setAutoThreshold("Otsu");
9  setOption("BlackBackground", true);
10 run("Convert to Mask");
11
12 //Analyze
13 run("Set Measurements...", "area centroid shape redirect=None decimal=3");
14 run("Analyze Particles...", "size=0-Infinity display clear");
15
```

The ‘run’ function

- You may have observed a pattern in the first macro: the run function is called multiple times. And all the parameter(s) of this function is string.
- The first parameter is the name of the process called, such as Convert to Mask.
- In the case the name ends with ..., this means that this function require some additional parameters, which are describe in the second string, such as sigma=3 for the Gaussian Blur... by example.
- In this case, string manipulation, and mainly concatenation, is very important to be able to dynamically modify some parameters.

```
run("Convert to Mask");
```

```
run("Gaussian Blur...", "sigma=3");
```

Exercise 1: Make a Simple Montage

- Open an Image (2_22.tif for example)
- Record the following actions:
  - Do a Z Projection, Max Intensity (Image > Stacks > Z-Project…)
  - Rename the window (right click, Rename…)
  - Adjust Contract & Brightness (Image > Adjust > Brightness/Contrast…)
  - Give a colour (Image > Lookup Table > Cyan)
  - Repeat the last 2 points after changing Colour
  - Make a Montage (Image > Stacks > Make Montage…)
  - Create the Macro

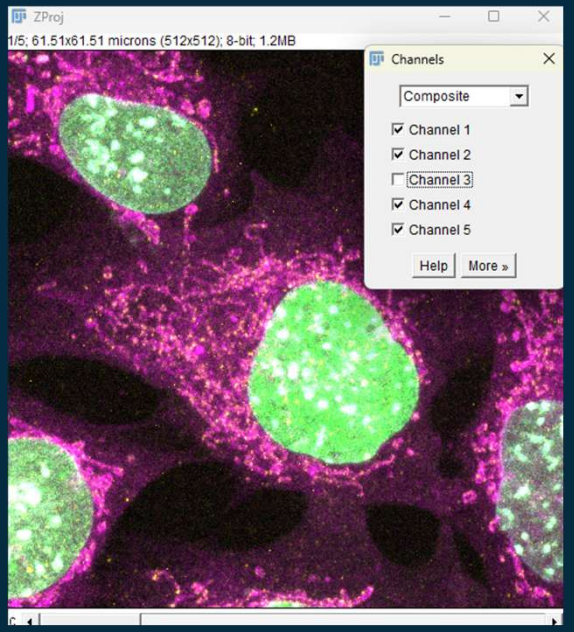
  Does EVERY actions get recorded? Try your Macro on another Image!

Colour: Composite Image

You can make composite image with the Image > Color > Channels Tools…

Select ‘Composite’ and unselect Channel 3.

Then More, Create RGB to get flatten image, then you can add a scale bar
Analysis > Tools > Scale Bar…



Window Selection

Image operation always happen on the **currently selected image.**

It is possible to select window with the ‘selectImage(“image_name”)’ function.

Some Image operation will create a new Image (Z Projection).

Other will apply the operation in place (Gaussian Blur).

In case of doubt, duplicate the image (Right click, Duplicate…)

Image Type

- Image type is indicated at the top left of the image

Some operations can only be made on some type!
8 and 16 bit are integer
32 bit are float
RGB are for display and saving

Copy/Paste Slice is possible between identical image type

Exercise 2: Make a Better Montage

- Open an Image (2_22.tif for example)
- Starting from the previous macro result
- Record the following actions:
  - With the Channel tools, make a composite image with only the fluorescent channels
  - Convert the image as RGB, add a scale bar, and copy it Edit > Copy
  - Select the Zprojection
  - Add a slice to it Image > Stacks > Add Slice and paste the composite Edit > Paste
  - Make the Montage
  - Save as PNG File > Save As > PNG

Variables: Getting Name

- To keep track of your images in the macro, it is important to rename, but also getting the name of the currently selected image

getTitle(); return the name of the currently open image

Img_name = getTitle();

It is possible to get much more than the title if required. Try the ‘get’ and check the autocompletion!

Exercise 3: Keep montage!

- Open an Image (2_22.tif for example)
- Starting from the previous macro result
- Modify the macro to:
  - Get the title of the initial image
  - Print it in the log window
  - When saving the montage as PNG, adjust the original name

Run your macro on multiple image and be sure that the montage image name is unique (not erased)

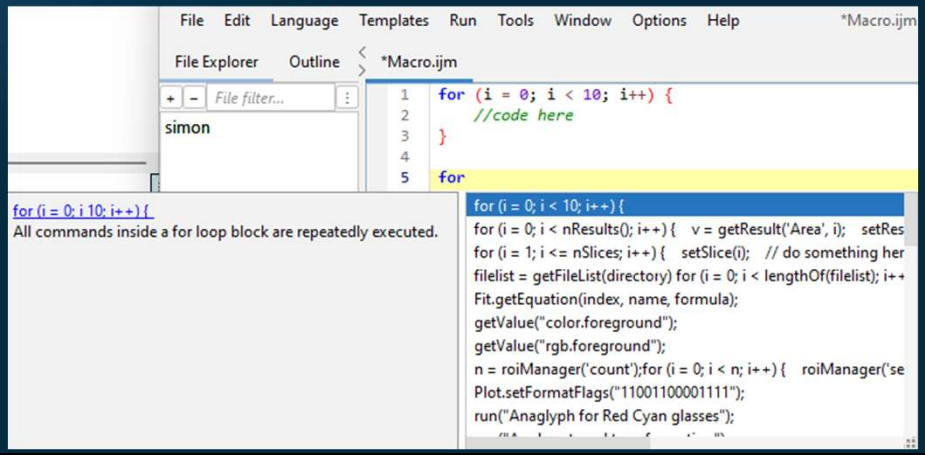
Programming: For Loop

- This can be translated as 'Do this job X times', or for example 'Stir the pot 10 times'.
- In the ImageJ Macro language, it requires a little more information:
  - An initialization stage (where we start, at 0 stir)
  - A condition (when to stop, at 10 stir)
  - An increment/decrement (how to change each time, 1 stir at a time)
  - The code to be executed in each iteration (stir)

Programming: For Loop

```
for (i = 0; i < 10; i++) {
        print(i)
}
```

If the condition is always false = infinite loop!



Exercise 4: Running the Macro through all images

- Starting from the previous macro result
- Modify the macro to:
  - Loop through all files within a directory – check ‘for’ and autocompletion!
    - You will need to assign a directory variable
  - Include the previously written code to make the montage
  - Open the image using the Bio-Format Importer Plugin > Bio-Format

Test the macro and note the issue(s)

Programming: if condition

- Check if the condition is true

```
if (a == b) {
        print(a+' is equal to '+b);
}
else {
         print(a+' is NOT equal to '+b);
}
```

Programming: if condition

It is possible to chain condition check with the keyword else if

It is possible to check multiple condition at the same time with && (and), and || (or)

Some function return a boolean (true/false) by default:

is(« grayscale »);

Exercise 5: Running the Macro through tif images

- Starting from the previous macro result
- Modify the macro to:
  - Check if the file that we will open is a tif
    - If it is, process as we have been doing
    - Else go to the next image

Programming: testing

It is important to test that the macro is working on multiple but similar images

```
run(“Close All”);
print("\\Clear");
run("Clear Results");
```

To start a macro with a clean slate

Programming: Array

The macro language can store data in Array

```
color_array = newArray("Cyan", "Yellow", "Grays", "Green",
"Magenta");
```

It is possible to loop through an Array with a for loop, or modify and specific entry by knowing its index

```
color_array[0] = “Blue”
```

Check the ‘Array’ autocompletion for more information

Programming: GUI

The macro language also allow you to build a full GUI with interactive window.

But before that, there is 2 interesting functions:

getDirectory("Choose directory"); // Open a GUI window to prompt the user to select a directory

waitForUser(string); // Stop the macro and wait for the user to click on ‘OK’ before proceeding.

Programming: Batch processing

Displaying images has a computational cost, which can slow down or even freeze Window!

It is possible to hide all images to avoid this extra cost by using:

- **setBatchMode("hide")**: Hide active image
- **setBatchMode("show")**: Display active image
- **setBatchMode("exit and display")**: Display ALL images

Optional Exercise: More general Montage

- Starting from the previous macro result
- Modify the macro so that:
  - The Lookup Table section is more polyvalent
    - getDimensions
    - Stack.setChannel within a for loop
    - Adjust inactive channel for Grays
  - Add a ‘choose directory’ window and BatchMode
  - Super optional: Add a GUI to allow identification and colour selection of the brightfield channel

Thanks for your attention